
Projeto Prático

Sistemas

Operativos

Licenciatura em Engenharia de
Telecomunicações e Informática

2024/2025

Da autoria de:

Luís Baptista A105419

José Martins A98556

GRUPO 10



Universidade do Minho

Índice

1.	Introdução	3
2.	Requisitos	3
3.	Comunicar entre Processos	4
4.	Implementação dos Programas	4
5.	Valorizações	5
6.	Conclusão	6
7.	Anexos	6
	Anexo 1: Estrutura dos Ficheiros Binários	6
	Anexo 2: Estrutura de Dados das Estatísticas (region_stats)	7
	Anexo 3: MakeFile	7

1. Introdução

Este relatório explica o desenvolvimento de uma solução para processar e analisar dados de medições de temperatura recolhidos por sensores em várias regiões do país. O principal objetivo é criar três programas que permitam ordenar os dados, calcular estatísticas e gerar relatórios a partir de um ficheiro binário com as medições.

Dado o grande volume de dados, é importante garantir um processamento eficiente, considerando as limitações de memória. Por isso, o trabalho foi feito no sistema operativo Linux, usando pipes anónimos e processos paralelos para melhorar o desempenho e facilitar a comunicação entre os programas.

O relatório vai explicar como foi abordado o problema, as escolhas feitas para a arquitetura dos programas, e o funcionamento de cada um dos três programas recomendados para a resolução deste problema:

- **Sort:** Ordena os dados de uma região
- **Stats:** Calcula as estatísticas como média, mediana, valor máximo e mínimo.
- **Report:** Cria um relatório com os dados de todas as regiões

2. Requisitos

O projeto exige a implementação de três programas distintos, como enunciado anteriormente, cada um com funcionalidades distintas:

- Sort: Ordena os registos de uma região de forma eficiente, utilizando bubble sort (linhas 16-24 no ficheiro sort.c) e escrever os registos ordenados de volta no ficheiro
- Stats: Calcula as estatísticas de uma região, usando o sort (linha 108 no ficheiro stats.c) para garantir previamente a sua ordenação
- Report: Apresenta as estatísticas de todas as regiões, utilizando pipes anónimos para comunicação entre processos e limitando o número de processos simultâneos (linhas 96-99 no ficheiro report.c) para evitar sobrecarga

-
- **Main:** Gera os dados iniciais de temperatura e coloca-os num ficheiro binário (Linhas 19-20 no ficheiro main.c)

3. Comunicar entre Processos

Os programas foram pensados e sucessivamente criados para se comunicarem de maneira eficiente, mesmo com grandes volumes de dados:

sort.c: Ordena registos diretamente no ficheiro binário, sem programas intermediários.

stats.c: Executa o sort via 'execd' antes de calcular as estatísticas, garantindo que os dados estejam ordenados.

report.c: Coordena processos paralelos, utilizando pipes para transferir dados entre os processos e implementa um mecanismo para limitar o número de processos simultâneos (max_processes), assegurando a estabilidade em sistemas com recursos limitados.

main.c: Implementa a escrita inicial dos dados no ficheiro binário, gerando medições de temperatura aleatórias para cada região.

4. Implementação dos Programas

A implementação dos programas em caso, já especificados, foi feita de forma modular, usando funções separadas para facilitar a leitura do código

main.c - Gerar um ficheiro binário com dados de medições de temperatura para diversas regiões

Função_principal - generate_temperature_data

A função gera medições de temperatura aleatórias e as guarda-as num ficheiro binário com o seguinte formato:

- Cabeçalho contendo o número de regiões (num_regions) e o número de registos por região (num_records)
- Registos de temperatura organizados por região.

Para executar basta escrever o seguinte comando substituindo < ... > pelo parâmetros que nos forem favoráveis: `./ generate_sensor_data <num_regioes> <num_registos> <ficheiro_saida>`

sort.c - Ordenar os registos de temperatura de uma região usando o método do *Bubble sort* direto no ficheiro binário.

Para executar basta escrever o seguinte comando substituindo < ... > pelo parâmetros que nos forem favoráveis: `./sort <ficheiro_binario> <regiao>`

stats.c - Calcular estatísticas de uma região executando o sort com o auxílio da função `execl`.

Output - As estatísticas podem ser escritas num ficheiro ou enviadas para o `stdout`.

Para executar basta escrever o seguinte comando substituindo < ... > pelo parâmetros que nos forem favoráveis: `./stats <ficheiro_binario> <regiao> <output>`

report.c - Apresentar as estatísticas de todas as regiões.

- Utiliza pipes para comunicação e limita os processos.

Para executar basta escrever o seguinte comando substituindo < ... > pelo parâmetros que nos forem favoráveis: `./report <num_regioes> <max_processes> <nome_ficheiro>`

5. Valorizações

Durante o desenvolvimento do projeto, foram implementadas algumas funcionalidades extra (recomendadas pelos docentes que melhoram o desempenho e a flexibilidade dos programas, tais como:

- Evitar ficheiros temporários no programa stats adicionando um novo argumento que permite que o output seja enviado diretamente para o `stdout`, eliminando a necessidade de criar ficheiros temporários. Isto melhora a execução do programa, principalmente quando o número de regiões é elevado.

-
- Limitação do número de processos simultâneos no programa report, incluindo o parâmetro `max_processes` (limita o número de processos que podem estar a calcular estatísticas em simultâneo). Em sistemas com pouca memória, esta aplicação ajuda a evitar sobrecargas e garante que o sistema continue a funcionar de forma estável, mesmo durante o processamento de muitas regiões.

Nota: Estas melhorias tornam o sistema mais robusto e eficiente, permitindo uma gestão mais eficaz dos recursos e garantindo que os dados sejam processados de forma rápida e sem grandes perdas de desempenho.

6. Conclusão

Em suma, este projeto demonstrou o uso eficiente de recursos em sistemas operativos para processar grandes volumes de dados. As técnicas de comunicação entre processos, como pipes, e a execução paralela permitiram criar uma solução robusta e adaptável. As valorizações implementadas foram um pequeno desafio inicialmente, mas com criatividade e esforço, foi possível que se tornassem parte deste projeto e desta forma consequentemente tornarem o sistema mais eficiente e pronto para aplicações reais.

Deste modo, este trabalho reflete a comunicação entre processos e o uso eficiente de recursos em sistemas com restrições, como memória limitada.

7. Anexos

Anexo 1: Estrutura dos Ficheiros Binários

1. Cabeçalho
 - Número de regiões (int)
 - Número de registos por região (int)
2. Dados - Registos de temperatura de todas as regiões
 - Cada região tem o mesmo número de registos e os dados estão organizados em blocos contínuos no ficheiro.

Anexo 2: Estrutura de Dados das Estatísticas (region_stats)

```
typedef struct region_stats {  
    int region_id; int median; float average; int max; int min; } region_stats;
```

Anexo 3: MakeFile

```
CC = gcc
```

```
CFLAGS = -Wall -g
```

```
all: generate_sensor_data sort stats report
```

```
generate_sensor_data: generate_sensor_data.c
```

```
    gcc -o generate_sensor_data generate_sensor_data.c main.c
```

```
sort: sort.c
```

```
    gcc -o sort sort.c
```

```
stats: stats.c
```

```
    gcc -o stats stats.c
```

```
report: report.c
```

```
    gcc -o report report.c
```

```
clean:
```

```
    rm -f sort stats report
```